College Helpdesk AI
Project Report

August 2026

Abstract

This report documents the College Helpdesk AI project, including architecture, tools, flowcharts, project structure, and working flow. The system is a local student-facing helpdesk assistant built with Python, Streamlit, and a lightweight knowledge retrieval pipeline.

Contents

1 Approach Comparison

This project uses a hybrid architecture combining local rule-based intent detection with Ollama LLM support for intent classification and answer generation. It is designed to remain deterministic while benefiting from LLM-assisted natural language handling.

The system combines:

- rule-based intent detection in `intents/classifier.py`,
- Ollama LLM-based classification and response generation in `rag/llm.py`,
- modular routing in `router.py`,
- targeted service modules in `tools/`, and
- local markdown knowledge lookup in `knowledge/`.

Alternative approaches considered:

- **LLM-based conversational AI:** more flexible and natural, but external service dependency and potential hallucinations would reduce reliability.
- **Static FAQ-only system:** simple and low-risk, but it cannot support transactional actions such as ticket creation or password reset.
- **Hybrid rule-based + retrieval:** chosen for balanced reliability, local execution, and multi-purpose workflow support.

Context and Problem

College students need a unified support channel for campus services and information. Existing campus support is fragmented across noticeboards, emails, and office visits. This project brings complaint handling, knowledge lookup, and password support into a single conversational interface.

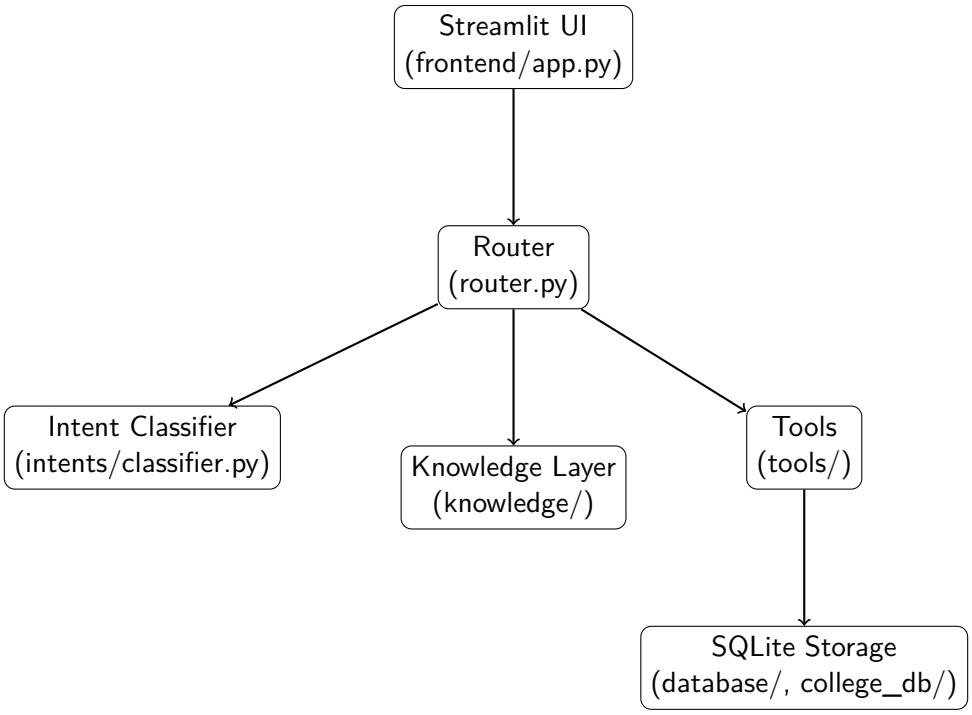
2 System Architecture

The architecture is built around a layered request pipeline.

Core components

- `frontend/app.py`: Streamlit UI and session management.
- `router.py`: Query router and intent dispatcher.
- `intents/classifier.py`: Rule-based intent detection.
- `tools/`: Auth, ticket, password reset, and contact lookup helpers.
- `knowledge/`: Local campus knowledge files.
- `rag/`: Ollama LLM and retrieval support for classification and answer generation.
- `database/` and `college_db/`: SQLite storage for users, tickets, and retrieval artifacts.
- `evaluation/`: Validation scripts and test cases.

Architecture Diagram



Request handling flow

1. Student logs in through the Streamlit interface.
2. The chat query is submitted to the router.
3. The router classifies intent and selects the right service path.
4. The selected module executes the action or returns a knowledge answer.
5. The response is returned to the UI.

Project structure

Path	Description
`frontend/app.py`	Streamlit chat interface and session flow
`router.py`	Main request router and intent dispatcher
`intents/classifier.py`	Rule-based intent detection
`tools/auth.py`	Login and user lookup
`tools/ticket_tool.py`	Create and delete tickets
`tools/password_reset.py`	Password update helpers
`tools/contact_tool.py`	Contact lookup helper
`knowledge/`	Local college information files
`database/`	SQLite database module and files
`README.md`	Project setup and usage guide
`project_report.tex`	LaTeX version of the project report
`project_report.md`	Markdown version of the project report
`.gitignore`	Excluded local files and environment artifacts

3 Test Results

The advanced evaluation script `evaluation/evaluate`$_a$*dvanced.pywasexecutedagainst*`evaluation/advanced`$_t$

`Total test cases: 50`

`Correct predictions: 50`

`Incorrect predictions: 0`

`Accuracy: 100.00%`

Category performance:

- `rag`: 14/14 (100.00%)
- $\texttt{password}_{r}eset$: 6/6(100.00%)
- $\texttt{create}_{t}icket$: 8/8(100.00%)
- $\texttt{view}_{t}ickets$: 8/8(100.00%)
- $\texttt{contact}_{o}ffice$: 6/6(100.00%)
- `greeting: 4/4 (100.00%)`
- `tricky: 4/4 (100.00%)`

These results confirm that the classifier and router correctly route evaluated user requests to the appropriate tool or knowledge path.

4 Challenges Faced

- Distinguishing ambiguous wording between knowledge queries and ticket creation.
- Ensuring correct tool selection for password reset and contact lookup flows.
- Designing a broad evaluation dataset that covers edge cases and sensitive actions.
- Maintaining reliable local knowledge answers without external LLM support.
- Keeping password reset and ticket management secure within a scoped prototype.

5 Future Improvements

- Add password hashing and stricter validation for password reset.
- Expand the evaluation dataset to cover more multi-turn and context-dependent interactions.
- Improve knowledge retrieval with semantic or vector search techniques.
- Add human evaluation of response quality in addition to routing accuracy.
- Enhance the Streamlit UI with clearer guidance, prompts, and next-step suggestions.
- Add automated end-to-end tests and deployment support.